Università degli Studi di Padova

DEPARTMENT OF MATHEMATICS

Master's Degree in Data Science

Optimization Methods for Portfolio Problem using Frank-Wolfe and its variants

Academic Year: 2024-25

Contents

1 Abstract **2**

2 Introduction to the Problem: Markowitz Portfolio Problem **2**

3 The Classical Frank-Wolfe Algorithm (FW) **2**

4 Frank-Wolfe Algorithm Variants **3**
4.1 Away-Steps Frank-Wolfe Method(AFW) 4
4.2 Pairwise Variant (PFW) 5

5 Duality Gap and Convergence Analysis **6**
5.1 Convergence Theory: Primal and Primal-Dual Guarantees 6

6 Optimality in Terms of Sparsity **7**

7 Application to Portfolio optimization **7**
7.1 Datasets 7
7.2 Experimental results 7
7.2.1 12 Industry Portfolios 8
7.2.2 17 Industry Portfolios 8
7.2.3 49 Industry Portfolios 8
7.3 Plot visualizations 9
7.3.1 Plots 9
7.3.2 Portfolio Composition 10

8 Conclusions **11**

9 References **11**

1 Abstract

This document explores the application of the Frank-Wolfe algorithm and its variants within the framework of portfolio optimization, with a specific focus on determining optimal investment allocations across a set of assets. The Frank-Wolfe method, known for its projection-free property and effectiveness in handling constrained convex optimization, is particularly suitable for problems requiring sparse or structured solutions. We analyze the Frank-Wolfe classic algorithm and the variants of the algorithm, the Away-steps Frank-Wolfe (AFW), and the Pairwise Frank-Wolfe (PFW), comparing them to better understand their behavior in estimating asset weights in a portfolio. Then, we investigate how different variants influence convergence, sparsity, and computational efficiency, offering insights into their practical use for large-scale financial optimization tasks.

2 Introduction to the Problem: Markowitz Portfolio Problem

The portfolio optimization problem was first introduced by Harry Markowitz in 1952, laying the foundation for modern portfolio theory. The central idea is to construct a portfolio, a distribution of wealth among a set of available assets, that balances maximizing expected return against minimizing financial risk. This problem is typically formulated as a constrained utility maximization problem, where risk is penalized via a risk-aversion parameter.

The classic portfolio problem, assumes that an investor allocates wealth among n available assets.
Let x_i denote the quantity of money invested in the i-th asset over a specified period, and r_i represent its corresponding return, which is modeled as a random variable with mean $\overline{r}$ and a covariance matrix Σ.

With a risk-adversion parameter $\gamma > 0$, which represents the investor's tolerance for risk, the portfolio optimization problem becomes a convex quadratic program:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \gamma x^T \Sigma x - \overline{r}^T x \\ \text{where} \quad & e^T x = 1 \\ & x \geq 0 \end{aligned}$$

Based on this, the portfolio's expected return is given by $\overline{r}^T x$, and its variance by $x^T \Sigma x$.
The problem is subject to two primary constraints:

- **Non-negativity constraint**: $x_i \geq 0$ for all $i = 1, \ldots, n$. This implies that short selling (selling assets one does not yet own) is not permitted.
- **Budget constraint**: $\sum_{i=1}^{n} x_i = B$. This ensures that the total amount of money invested equals a predefined budget B, which can typically be normalized to 1.

The primary goal is to find a set of asset weights (x) that minimizes the portfolio's variance (risk) while simultaneously maximizing its expected return, all while satisfying the budget and non-negativity constraints.

3 The Classical Frank-Wolfe Algorithm (FW)

The original Frank-Wolfe algorithm *(algorithm 1)*, introduced in 1956, it's an iterative first-order optimization algorithm to solve quadratic programming problems with linear constraints and it's also known as *conditional gradient method* or *reduced gradient method*. FW relies on a simpler linear optimization subproblem at each iteration. Recently it has seen a revival due to its nice properties compared to projected gradient methods, in particular for machine learning applications, since:

- FW is a "projection-free" algorithm and with this feature bypasses the often computationally prohibitive projection steps required and it has a much smaller cost per iteration compared to the one we have for projected gradient
- the iterates keep a desirable structure (like, e.g. sparsity)

We consider a general constrained convex optimization problem of the form

$$\min_{x \in \mathcal{D}} f(x),$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is convex with Lipschitz-continuous gradient, and $\mathcal{D} \subseteq \mathbb{R}^n$ is a compact convex set.

The process begins by selecting an initial feasible point $x^{(0)} \in \mathcal{D}$. It then proceeds for a fixed number of iterations, $k = 0, 1, \ldots, K$. At each iteration, the algorithm first solves a linear subproblem to find a direction, or "atom," $s^{(k)}$, that minimizes the linearized objective function:

$$s^{(k)} := \arg\min_{s \in \mathcal{D}} \left\langle s,\, \nabla f(x^{(k)}) \right\rangle.$$

Using a step size $\gamma_k = \frac{2}{k+2}$, the solution is then updated by taking a convex combination of the current point and the new direction:

$$x^{(k+1)} = (1 - \gamma_k)\, x^{(k)} + \gamma_k\, s^{(k)}.$$

This ensures each iterate $x^{(k)}$ lies in $\mathcal{D}$ and is a convex combination of previously selected extreme points ("atoms"), adding at most one new atom per iteration.

Algorithm 1 Frank–Wolfe Method

```
let x^(0) ∈ D
for k = 0, 1, 2, ..., K do
    s^(k) = arg min_{s∈D} ⟨s, ∇f(x^(k))⟩
    Update   x^(k+1) := (1 − γ) x^(k) + γ s, for γ := 2/(k+2)
end for
```

The Linear Minimization Oracle (LMO)

The **LMO** is the internal process used to compute the Frank-Wolfe direction at each iteration.

Formally, given the current iterate $x^{(t)}$ and the gradient $\nabla f(x^{(t)})$, the oracle solves the linear subproblem:

$$s_t := \arg\min_{s \in \mathcal{D}} \langle \nabla f(x^{(t)}), s \rangle$$

The solution s_t identifies an extreme point of the feasible region $\mathcal{D}$ that is most aligned with the direction of steepest descent in the linearized objective.

The LMO does not require projections or full gradient steps: it simply returns a vertex of $\mathcal{D}$, which becomes the target for the next convex update. Its simplicity and efficiency are key to the practical performance of the Frank-Wolfe algorithm, especially when the feasible set $\mathcal{D}$ has combinatorial or polyhedral structure.

4 Frank-Wolfe Algorithm Variants

To address specific limitations of the classical Frank-Wolfe (FW) method, most notably its sublinear convergence and the tendency for zig-zagging near the boundary of the feasible set, several algorithmic variants have been developed. These enhancements aim to improve either the theoretical rate of convergence or the sparsity and quality of the solutions obtained in practice.
The zig-zagging phenomenon typically arises when the optimal solution x^* lies on the boundary of the feasible region D In such cases, the standard FW algorithm tends to alternate between vertices that define the face containing x^*. This behavior leads to inefficiency, especially in high-dimensional problems or when high-precision solutions are required.

Modern variants incorporate mechanisms that either allow movement away from unproductive directions or enable corrective steps that more directly approach optimality. These include:

- **FW with Approximate Linear Subproblems:** In scenarios where solving the linear minimization oracle (LMO) exactly is computationally expensive, this variant permits the use of approximate solutions. At each iteration k, instead of solving the linear subproblem:

$$s^{(k)} := \arg\min_{s \in \mathcal{D}} \langle s, \nabla f(x^{(k)}) \rangle,$$

one can find an approximate solution s satisfying:

$$\langle s, \nabla f(x^{(k)}) \rangle \leq \min_{\hat{s} \in \mathcal{D}} \langle \hat{s}, \nabla f(x^{(k)}) \rangle + \frac{1}{2} \delta \gamma C_f,$$

where $\delta \geq 0$ controls the approximation quality, and $\gamma = \frac{2}{k+2}$ is the step size. This controlled relaxation enables convergence to be preserved even when only approximate solutions are feasible, an advantage for large-scale or structured optimization problems where the LMO is computationally costly.

- **Line Search for the Step-Size:** Instead of relying on a predefined diminishing step-size (e.g., $\gamma_k = \frac{2}{k+2}$), this variant dynamically chooses the step-size at each iteration to minimize the objective along the update direction:

$$\gamma := \arg\min_{\gamma \in [0,1]} f(x^{(k)} + \gamma(s - x^{(k)})).$$

This approach typically leads to faster descent and better empirical performance, especially when the curvature of the objective function varies across the domain (i.e., in ill-conditioned problems). The adaptive nature of this strategy makes each update more efficient by fully exploiting the local geometry of the objective.

- **Fully Corrective Variant:** This variant enhances sparsity and convergence by re-optimizing the solution over the convex hull of all previously used atoms. After computing the new direction $s^{(k)}$, instead of a simple convex combination update:

$$x^{(k+1)} := (1 - \gamma) x^{(k)} + \gamma s^{(k)},$$

the solution is recomputed as:

$$x^{(k+1)} := \arg\min_{x \in \mathrm{conv}(s^{(0)}, \ldots, s^{(k+1)})} f(x),$$

where $\mathrm{conv}(\cdot)$ denotes the convex hull. This means solving a (typically small-scale) convex optimization subproblem over all directions found so far. Although more computationally demanding per iteration, this strategy can improve sparsity (fewer active atoms) and convergence speed, especially when high-accuracy or low-rank solutions are desired.

4.1 Away-Steps Frank-Wolfe Method(AFW)

The *Away-Steps* variant of the FW method *(algorithm 2)*, proposed by Wolfe (1970), addresses the zig-zagging behavior described before. The core idea is, to not only move towards a new "good" atom, as in classical FW, but also to potentially move *away* from an "old" atom that is currently "bad" with respect to the objective function. This helps to:

- **Mitigate Zig-Zagging**: By allowing movement away from vertices, the algorithm can escape the oscillatory behavior often seen near the boundary of the feasible set.
- **Improve Sparsity**: It can lead to iterates represented by even fewer active atoms, similar to the fully-corrective variant.
- **Achieve Faster Convergence**: For certain problem classes, away-steps can yield faster (e.g., linear) convergence rates. This requires representing the current iterate as a convex combination of the current atoms, allowing for the removal of an atom.

Algorithm 2 Away-steps FW method

1: Let $x^{(0)} \in A$, and $S^{(0)} = \{x^{(0)}\}$
2: **for** $t = 0$ to T **do**
3: Let $s_t = \mathrm{LMO}_A(\nabla f(x^{(t)}))$ and $d_t^{FW} = s_t - x^{(t)}$
4: Let $v_t \in \arg\max_{v \in S^{(t)}} \langle \nabla f(x^{(t)}), v \rangle$ and $d_t^A = x^{(t)} - v_t$
5: **if** $g_t^{FW} = -\langle \nabla f(x^{(t)}), d_t^{FW} \rangle \leq \epsilon$ **then**
6: **return** $x(t)$
7: **end if**
8: **if** $-\langle \nabla f(x^{(t)}), d_t^{FW} \rangle \geq -\langle \nabla f(x^{(t)}), d_t^A \rangle$ **then**
9: $d_t = d_t^{FW}$, $\gamma_{\max} = 1$
10: **else**
11: $d_t = d_t^A$, $\gamma_{\max} := \frac{\alpha_{v_t}}{1-\alpha_{v_t}}$
12: **end if**
13: Line-search: $\gamma_t \in \arg\min_{\gamma \in [0, \gamma_{\max}]} f(x^{(t)} + \gamma d_t)$
14: Update $x^{(t+1)} = x^{(t)} + \gamma_t d_t$
15: Update $S^{(t+1)} = \{v \in A \mid \alpha_v^{(t+1)} > 0\}$
16: **end for**

The away direction d_t^A is defined by finding the atom v_t in $S^(t)$ that maximizes the potential of descent given by a $g_t^A = \langle -\nabla f(x)^{(t)}), x^{(t)} - v_t$.
This search is fundamentally easier than the linear oracle search LMO_D

Line 9 finds the maximum step-size and ensures that the new iterate $x^{(t)} + \gamma d_t^A$ stays in D. The conservative maximum step-size makes it possible not needing a powerful oracle. with the ability to know when the boundary of D is crossed along a specific line. This is the reason why AFW requires to maintain $S^{(t)}$ (unlike standard FW).
Finally, as in classical FW, the FW gap g_t^{FW} is an upper bound on the unknown suboptimality, and can be used as a stopping criterion:

$$g_t^{FW} = \langle -\nabla f(x^{(t)}), d_t^{FW} \rangle \geq \langle -\nabla f(x^{(t)}), x^* - x^{(t)} \rangle \geq f(x^{(t)}) - f(x^*) \quad \text{(by convexity)}$$

4.2 Pairwise Variant (PFW)

The Pairwise Frank Wolfe method *(algorithm 3)* refines the FW method by restricting each update to a transfer of weight between two atoms only: from the away atom v_t to the the FW atom s_t, and keeps all other α weights unchanged.
More specifically, at each iteration, PFW performs a *pairwise step*, adjusting the weights as follows: $\alpha_{v_t}^{(t+1)} = \alpha_{v_t}^{(t)} - \gamma$, and $\alpha_{s_t}^{(t+1)} = \alpha_{s_t}^{(t)} + \gamma$, for some step-size $\gamma \leq \gamma_{\max} := \alpha_{v_t}^{(t)}$. This stands in contrast to classical Frank-Wolfe, which rescales all active weights at each step. The pairwise strategy allows for more localized and potentially sparser updates, which can lead to faster convergence in certain settings.

Algorithm 3 Pairwise FW method

1: Let $x^{(0)} \in A$, and $S^{(0)} = \{x^{(0)}\}$
2: **for** $t = 0$ to T **do**
3: Let $s_t = \mathrm{LMO}_A(\nabla f(x^{(t)}))$ and $d_t^{FW} = s_t - x^{(t)}$
4: Let $v_t \in \arg\max_{v \in S^{(t)}} \langle \nabla f(x^{(t)}), v \rangle$ and $d_t^A = x^{(t)} - v_t$
5: **if** $g_t^{FW} = -\langle \nabla f(x^{(t)}), d_t^{FW} \rangle \leq \epsilon$ **then**
6: **return** $x(t)$
7: **end if**
8: $d_t = d_t^{PFW} = s_t - v_t and \gamma_{max} = \alpha_{v_t}$
9: Line-search: $\gamma_t \in \arg\min_{\gamma \in [0, \gamma_{\max}]} f(x^{(t)} + \gamma d_t)$
10: Update $x^{(t+1)} = x^{(t)} + \gamma_t d_t$
11: Update $S^{(t+1)} = \{v \in A \mid \alpha_v^{(t+1)} > 0\}$
12: **end for**

Although the convergence rate guarantees for the PFW variant are generally weaker than those for other FW variants, due to the lack of tight bounds on the number of problematic swap steps, it often performs very well in practice outperforming AFW, especially in the case of sparse solutions. This method is particularly effective at pruning inactive coordinates from the active set, unlike the away-step, which redistributes weight uniformly across all active atoms, potentially requiring additional corrective steps later, the pairwise step transfers weight directly onto the selected FW atom s_t, resulting in more targeted and efficient updates.

5 Duality Gap and Convergence Analysis

Analyzing the convergence behavior of Frank-Wolfe (FW) algorithms can be useful to asses their efficiency and suitability for large-scale convex optimization tasks such as portfolio selection. In this section, we summarize the theoretical convergence guarantees for the classical FW algorithm and its major variants, and introduce geometric and analytical concepts that underlie these results.

To formally characterize convergence rates, several important quantities are presented:

- **Duality Gap**: For any feasible point $x \in D$, the simple surrogate duality gap $g(x)$ is defined as:

  $$g(x) := \max_{s \in D} \langle x - s, \nabla f(x) \rangle$$

  This quantity serves as a *certificate* for the current approximation, as it provides an upper bound on the linearization $f(x) + \langle s - x, \nabla f(x) \rangle$, making sure that it always lies below the graph of the funcion f. The duality gap serves as a metric to determine how close the current solution is to the optimal one, making $g(x)$ an easily verifiable measure of progress and a robust stopping criterion.

- **Curvature Constant (C_f)**: The convergence analysis of Frank-Wolfe type algorithms relies on the *curvature constant* C_f: a measure of "non-linearity" of the objective function f over the domain D, so it can express the deviation of f at point y from the linear approximation given by the gradient at point x, normalized by the inverse of γ^2
  The curvature constant C_f of a convex and differentiable function $f : \mathbb{R}^n \to \mathbb{R}$ with respect to a compact domain D is defined as:

  $$C_f := \sup_{\substack{x,s \in \mathcal{D} \\ \gamma \in [0,1] \\ y = x + \gamma(s-x)}} \frac{2}{\gamma^2}(f(y) - f(x) - \langle y - x, \nabla f(x) \rangle)$$

  A bounded C_f ensures that the deviation of f from its linear approximation remains controlled, which is fundamental for the algorithm's convergence. This constant is closely related to the Lipschitz continuity of the gradient of f.

5.1 Convergence Theory: Primal and Primal-Dual Guarantees

The strength of the Frank-Wolfe framework is underpinned by robust convergence guarantees, even under conditions of approximate subproblem solutions or inexact gradients.
Lemma 5 (Improvement per Step): This fundamental lemma establishes the progress made in each iteration. For a step $x^{(k+1)} := x^{(k)} + \gamma(s - x^{(k)})$ with arbitrary step-size $\gamma \in [0, 1]$, it holds that:

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \gamma g(x^{(k)}) + \frac{\gamma^2}{2} C_f (1 + \delta)$$

This bound holds if s is an approximate linear minimizer satisfying $\langle s, \nabla f(x^{(k)}) \rangle \leq \min_{\hat{s} \in \mathcal{D}} \langle \hat{s}, \nabla f(x^{(k)}) \rangle + \frac{1}{2}\delta\gamma C_f$. This lemma is the cornerstone for proving the main convergence theorems.
Convergence in primal error: After $O\left(\frac{1}{\epsilon}\right)$ iterations, the iterate $x^{(k)}$ of any Frank-Wolfe variant provides an ϵ-approximate solution, i.e., $f(x^{(k)}) \leq f(x^*) + \epsilon$, where x^* is the optimal solution.

for e **do**ach $k \geq 1$, the iterates $x^{(k)}$ of any FW algorithm satisfy:

$$f(x^{(k)}) - f(x^*) \leq \frac{2C_f}{k+2}(1 + \delta)$$

where $x^* \in \mathcal{D}$ is an optimal solution and $\delta \geq 0$ is the accuracy of the internal linear subproblems. This result highlights an $O(1/k)$ convergence rate, which is known to be worst-case optimal for algorithms that add only one new atom per iteration.

Primal-Dual Convergence: This is an important result demonstrating that the FWalgorithms achieve a guaranteed small duality gap it those are run for $K \geq 2$ iterations, then there exists an iterate $x^{(\hat{k})}$, for $1 \leq \hat{k} \leq K$, with a duality gap bounded by:

$$g(x^{(\hat{k})}) \leq \frac{2\beta C_f}{K+2}(1+\delta)$$

where $\beta = \frac{27}{8} = 3.375$, and $\delta \geq 0$ is the accuracy to which the linear subproblems are solved, C_f is the curvature constant of the objective function.
The idea behind the proof strategy for the duality gap cannot remain large over multiple iterations, as the successive step improvements would eventually drive the solution below the optimal value.

6 Optimality in Terms of Sparsity

The Frank-Wolfe algorithm is proven to be worst-case optimal in terms of the sparsity of the obtained solutions. The number of used atoms (i.e., the sparsity of the solution) is $O(1/\epsilon)$ for a primal and/or dual approximation error of ϵ. This lower bound of $\Omega(1/\epsilon)$ matches the upper bound derived from the convergence analysis, characterizing the fundamental trade-off between sparsity and approximation quality for this class of optimization problems.

7 Application to Portfolio optimization

In this section, we apply several variants of the FW algorithm to the portfolio optimization problem with 3 real-world datasets. The methods considered include the classical FW algorithm, its version with exact line search and two enhanced variants: the away-step FW and the pairwise FW methods

7.1 Datasets

To empirically evaluate the performance of Frank-Wolfe algorithms in the context of portfolio optimization, we utilize a family of benchmark datasets from the Kenneth R. French Data Library. These datasets consist of value-weighted returns for U.S. industry portfolios, aggregated at varying levels of sector granularity. All datasets report value-weighted returns, and span multiple decades (from July 1926 onward). Specifically, we make use of the following:

- 12 Industry Portfolios: this dataset aggregates stock returns into 12 broad industry groups: Consumer Non-Durables, Consumer Durables, Manufacturing, Energy, Chemicals, Business Equipment, Telecommunications, Utilities, Shops, Healthcare, Money (Financials), and Other.
- 17 Industry Portfolios: a more detailed breakdown into 17 sectors, offering finer separation among industrial, business services, and consumer-related categories. This dataset includes Food, Mines, Oil, Clothes, Durables, Chemicals, Consumer, Construction, Steel, Fabricated Products, Machinery, Cars, Transportation, Utilities, Retail, Finance, and Other.
- 49 Industry Portfolios: This version contains daily returns for 49 industries, offering the most granular industry classification among the three datasets.

7.2 Experimental results

For our optimization problem we take into account these 3 (industry portfolios) dataset and for a better benchmark we consider just the period from 2001 to 2025.

7.2.1 12 Industry Portfolios

Table 1 Summarizes the results, indicating that Pairwise FW, FW Line Search and Away-Steps outperform FW diminishing in terms of both iteration count and runtime. Pairwise FW converges the fastest with only 4 iterations and a runtime of 0.0001s, while FW Line Search also shows strong performance with 8 iterations and the lowest optimality gap (2.89e-07) at the same minimal runtime. Away-Steps FW requires 9 iterations and a slightly higher runtime of 0.0002s, achieving a comparable optimality gap. In contrast, FW Diminishing is less efficient, taking 30 iterations and 0.0006s. Overall, all methods achieve optimality gaps on the order of 1e-07, confirming they can reach high solution accuracy across the board.

Algorithm	**Iterations**	**Final Gap**	**CPU Time (s)**
FW Diminishing	30	3.96e-07	0.0006
FW Line Search	8	2.89e-07	0.0001
Pairwise FW	4	6.31e-07	0.0001
Away-Steps FW	9	3.28e-07	0.0002

Table 1: Comparisons between different FW variants with 12 Industry Portfolios

7.2.2 17 Industry Portfolios

Table 2 illustrates the results obtained with the dataset with 17 Industry Portfolios. Away stands out as the most efficient method, converging in just 7 iterations and the lowest final gap. Also FW line search and Pairwise perform well, with PFW a bit better than FW, requiring only and 8 iterations, instead of 13. In terms of CPU Time they achieve same low runtimes (0.0002s). In contrast, FW Diminishing is noticeably slower, taking 40 iterations and 0.0005 seconds to converge, while also producing the largest optimality gap. Overall, FW, AFW and PFW proves they're the most effective algorithms on this dataset, with the last two (AFW,PFW) offering the best trade-off between convergence speed and solution accuracy.

Algorithm	**Iterations**	**Final Gap**	**CPU Time (s)**
FW Diminishing	40	8.70e-07	0.0005
FW Line Search	13	6.79e-07	0.0002
Pairwise FW	8	3.93e-07	0.0002
Away-Steps FW	7	3.660e-07	0.0001

Table 2: Comparisons between different FW variants with 17 Industry Portfolios

7.2.3 49 Industry Portfolios

On the higher-dimensional 49 Industry Portfolios dataset it is possible to observe in *table 3* distinctions in convergence efficiency. FW Diminishing is the least efficient, requiring 76 iterations and the longest runtime (0.0008s), while also producing a relatively large duality gap. In contrast, FW Line Search and Away-Steps FW both require 6 iterations, have the same running time (0.0001s) and final gaps (8.86e-07). Pairwise FW requires 8 iterations and the same running time, but the final gap is smaller: 2.19e-07. These results suggest again that Away-Steps FW and Pairwise FW show again their efficiency and also FW Line Search can perform quite well.

Algorithm	**Iterations**	**Final Gap**	**CPU Time (s)**
FW Diminishing	76	9.81e-07	0.0008
FW Line Search	6	8.86e-07	0.0001
Pairwise FW	8	2.19e-07	0.0001
Away-Steps FW	6	8.86e-07	0.0001

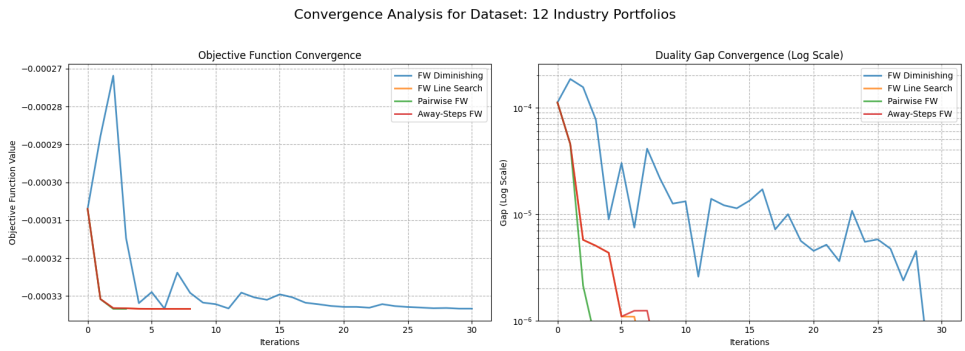
Table 3: Comparisons between different FW variants with 49 Industry Portfolios

7.3 Plot visualizations

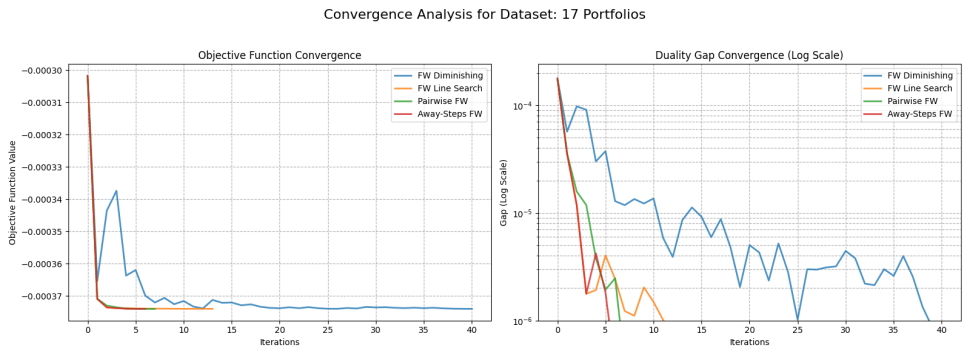
To complement the numerical evaluation, we present graphical analyses *(Figure 1)* for each dataset: 12, 17, and 49 Industry Portfolios, focusing on three key aspects: objective function convergence, duality gap convergence and the resulting optimal portfolio composition.

7.3.1 Plots

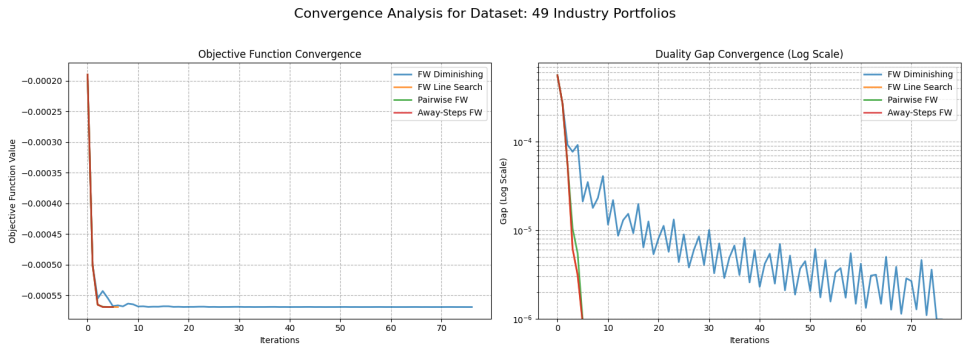
The following plots illustrate the objective value convergence over iterations and duality gap convergence for each method on datasets containing 12, 17, and 49 assets, respectively. In particular it is possible to note that the method using diminishing step size exhibits the zig-zagging behaviour described before, which is indicative of its slower convergence, especially in comparison to more adaptive methods like line search or FW-variants.



(a) Convergence for 12 Portfolios



(b) Convergence for 17 Portfolios



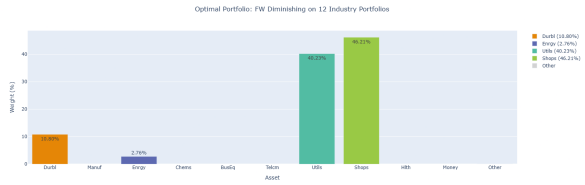
(c) Convergence for 49 Portfolios

Figure 1: Convergence plots for portfolio optimization problems of varying dimensionality.

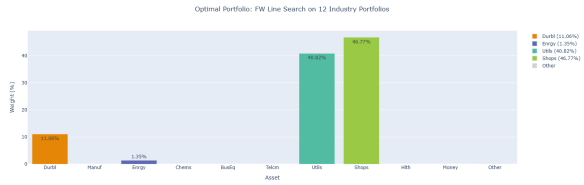
7.3.2 Portfolio Composition

This section presents bar plots that illustrate the asset selection for each optimization method. Each plot indicates the weight assigned to individual assets in the final portfolio, highlighting which assets are favored by different Frank-Wolfe variants.

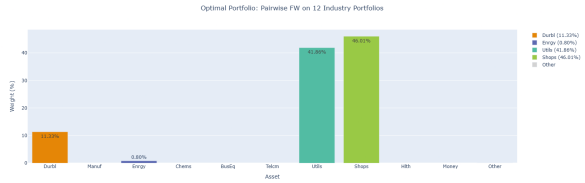
12 Industry final portfolio assets
All Frank-Wolfe methods identified the same subset of assets for inclusion in the optimal portfolio. While the specific weight allocations varied slightly across methods, the overall composition remained consistent and is made with: Durables, Business equity, Utilities and Shops



(a) FW Diminishing



(b) FW Linear search

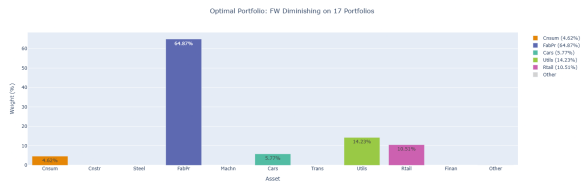


(c) FW Pairwise

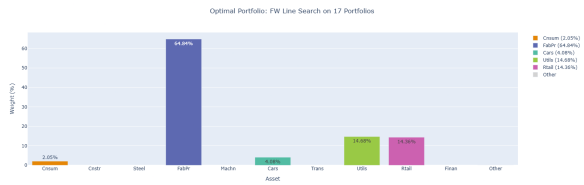
(d) FW Away steps

Figure 2: Portfolio weights selected by Frank-Wolfe variants on the 12-asset dataset.

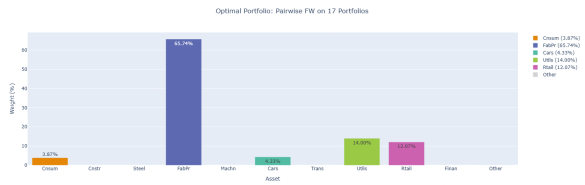
17 Industry final portfolios assets
For the 17-industry portfolio, all methods also selected the same assets in the final portfolio. As with the 12-asset case, the variations among the methods were limited to the precise weight assignments rather than the composition itself. The assets selected are: Fabricated Products, Cars and Retail.



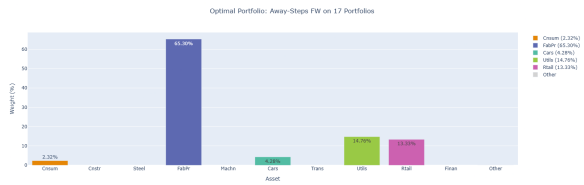
(a) FW Diminishing



(b) FW Line search



(c) FW Pairwise

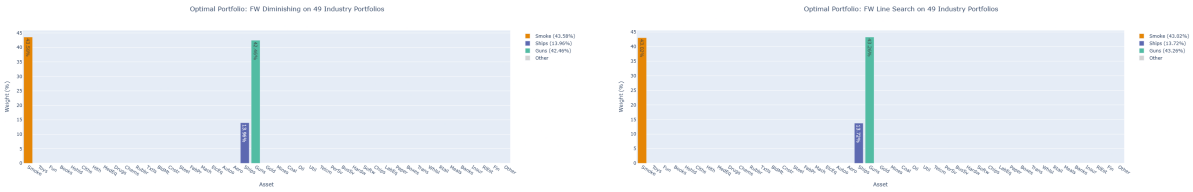


(d) FW Away steps

Figure 3: Portfolio weights selected by Frank-Wolfe variants on the 17-asset dataset.

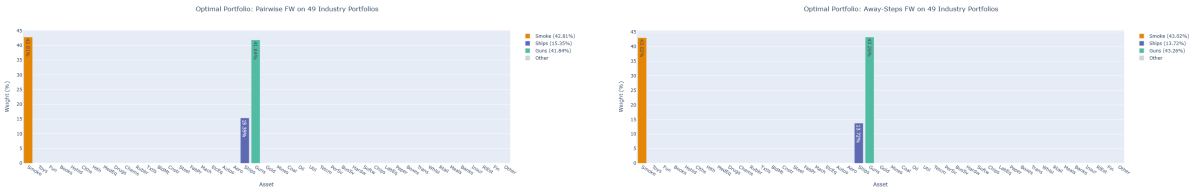
49 Industry final portfolio assets
Lastly, in the 49-industry portfolio case, all methods selected again the same set of assets, with only slight differences in the weight distributions. Despite the increased dimensionality, the consistency in asset selection across Frank-Wolfe variants suggests robustness in their ability to identify the most relevant

assets, since the main differences remained in how each method allocated proportions among the selected assets, rather than in the choice of assets themselves. The selected assets are. Smoke, Fun, Ships, Guns and Mines.



(a) FW Diminishing

(b) FW Line search



(c) FW Pairwise

(d) FW Away steps

Figure 4: Portfolio weights selected by Frank-Wolfe variants on the 49-asset dataset.

8 Conclusions

The aim of this project is to analyze the application of the Frank-Wolfe algorithm and its variants to portfolio optimization.
The results highlight the superiority of advanced variants over the classical approach, confirming the theoretical expectation that these methods are better suited and remain superior to overcome slow convergence in complex problems.
The FW Diminishing algorithm confirms its practical inefficiency, with sublinear convergence (O(1/t)) given by the "zig-zagging" phenomenon when the solution lies on the boundary of the domain. The Away-Steps FW and Pairwise FW variants overcome this weakness by introducing the use of an active-set, they can guarantee global linear convergence and prove to be, in practice, the fastest and most reliable methods.
However, no single method proves to be universally superior: the optimal choice depends on the problem's characteristics, in fact, a particularly interesting result is the good performance of FW with Line Search: it can be shown that it can achieve linear convergence for quadratic objectives when the optimum lies in the strict interior of the domain for strongly convex constraint sets and this suggests that when the optimal solution lies in a well-conditioned region rather than on a low-dimensional face of the domain.

9 References

1. M. Jaggi. *Revisiting Frank-Wolfe: Projection-Free Sparse Convex Optimization.* École Polytechnique.

2. S. Lacoste-Julien and M. Jaggi. *On the Global Linear Convergence of Frank-Wolfe Optimization Variants.*

3. I. M. Bomze, F. Rinaldi, and D. Zeffiro. *Frank–Wolfe and Friends: A Journey into Projection-Free First-Order Optimization Methods.*

4. C. W. Combettes and S. Pokutta. *Complexity of Linear Minimization and Projection on Some Sets*